DM2026 Assignment 2: Cross-Validation, SVM, Association Rules, and Clustering

Yu-Shun Liu
Student ID: 413551030

GitHub Repository: `https://github.com/RaisoLiu/DM2026-Assignment-2`

1 Q1: K-Fold Cross-Validation

Continuation of Assignment 1 `Real_World_Classification.ipynb`, using the updated `linear_model.py` from the course GitHub. The same preprocessing pipeline is reproduced: LabelEncoder on Species, KNNImputer($k = 5$), train/test split (70/30, seed=40), and MinMaxScaler (fit on training data only).

1.1 1(a) 5-Fold Cross-Validation: 4×4 Hyperparameter Grid

We perform 5-fold CV (`KFold(n_splits=5, shuffle=True, random_state=40)`) on the training set (350 samples) over 16 combinations of learning rate and L2 regularization strength. The model uses `n_iteration=10000`, `val_ratio=0.2` (an internal validation split used only for early stopping within each fold; it does not affect the outer CV evaluation, which always scores on the held-out fold).

Table 1: Mean ± standard deviation of 5-fold CV accuracy for each (lr, λ) combination

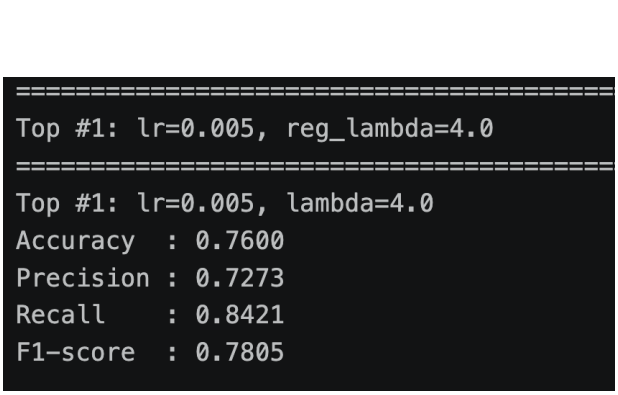
	$\lambda = 1.0$	$\lambda = 2.0$	$\lambda = 4.0$	$\lambda = 8.0$
lr=0.005	0.7200 ± 0.0420	0.7257 ± 0.0514	$\mathbf{0.7314 \pm 0.0377}$	0.7257 ± 0.0291
lr=0.01	0.7257 ± 0.0609	0.7286 ± 0.0527	0.7286 ± 0.0443	0.7229 ± 0.0265
lr=0.1	0.7171 ± 0.0388	0.7286 ± 0.0469	0.7257 ± 0.0464	0.7229 ± 0.0265
lr=0.5	0.7171 ± 0.0388	0.6886 ± 0.0481	0.5229 ± 0.0114	0.4657 ± 0.0280

1.2 1(b) Top 2 Hyperparameter Settings — Test Evaluation

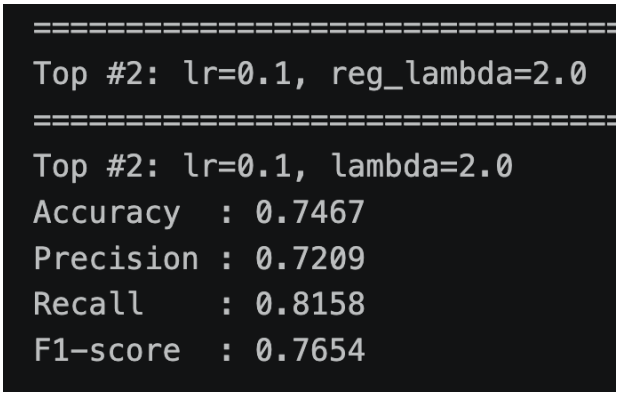
The top 2 settings by mean CV accuracy are retrained on the full training set and evaluated on the held-out test set (150 samples). Here, "training set" means the original 70% training split from Assignment 1 (350 samples), not one fold from cross-validation. The 5 folds are used only inside the training set for hyperparameter selection; the final test set is not used during CV. Note: the second-best CV accuracy (0.7286) is shared by three settings: (0.01, 2.0), (0.01, 4.0), and (0.1, 2.0). We select (0.1, 2.0) as #2 by choosing the setting with the largest learning rate (faster convergence) and smallest λ (least regularization).

Table 2: Test evaluation for top 2 hyperparameter settings

Rank	Setting	Accuracy	Precision	Recall	F1-score
#1	lr=0.005, λ=4.0	0.7600	0.7273	0.8421	0.7805
#2	lr=0.1, λ=2.0	0.7467	0.7209	0.8158	0.7654



(a) Top #1: lr=0.005, λ=4.0



(b) Top #2: lr=0.1, λ=2.0

Figure 1: Screenshots of `evaluate_binary_classifier` output for the top two hyperparameter settings.

1.3 1(c) Observations

- **Small learning rates are robust to regularization strength.** Learning rates 0.005, 0.01, and 0.1 achieve similar mean CV accuracy ($\sim$72–73%) across all λ values, but several differences are within one fold-level standard deviation. Therefore, the top ranking should be interpreted cautiously rather than as a large performance gap.
- **Large learning rate + strong regularization causes severe underfitting.** At lr=0.5, performance degrades sharply when $\lambda \geq 4.0$ (0.5229 ± 0.0114 and 0.4657 ± 0.0280). The low means indicate consistently poor performance across folds.
- **Good generalization from CV to test.** The top-1 setting (lr=0.005, λ=4.0) achieves 0.7314 ± 0.0377 CV accuracy and 76.0% test accuracy, and the top-2 setting generalizes similarly ($0.7286 \pm 0.0469 \rightarrow 74.7\%$). This confirms that 5-fold CV on this dataset provides a useful estimate of out-of-sample performance.
- The top-1 model shows higher recall (84.2%) than precision (72.7%), meaning it favors identifying positive samples at the cost of more false positives.

2 Q2: SVM Classification

We use the `mobile_price.csv` dataset (2000 samples, 20 features, 4 balanced classes). Data is split 60/20/20 into train/validation/test sets (`random_state=42`). We use `sklearn.svm.SVC` with an RBF kernel and γ = scale. This experiment keeps the raw feature representation fixed because it gives strong empirical validation/test performance. However, γ = scale is only a global

kernel-coefficient heuristic and does not replace per-feature standardization; standardizing features remains a reasonable additional preprocessing variant. F1-score uses `macro` averaging throughout.

2.1 2(a) SVM with C=1.0

Table 3: SVM (C=1.0) performance across data splits

Set	Accuracy	F1 (macro)
Train	0.9500	0.9505
Validation	0.9475	0.9463
Test	0.9625	0.9617

2.2 2(b) Performance vs. Regularization Parameter C

We sweep $C \in \{0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000\}$.

Table 4: SVM performance for different C values

C	Train Acc	Train F1	Val Acc	Val F1	Test Acc	Test F1
0.001	0.2625	0.1040	0.2325	0.0943	0.2300	0.0935
0.01	0.3242	0.2182	0.3075	0.2249	0.3000	0.2111
0.1	0.8942	0.8959	0.8950	0.8949	0.9000	0.8987
1	0.9500	0.9505	0.9475	0.9463	0.9625	0.9617
10	0.9692	0.9696	0.9450	0.9442	0.9700	0.9693
100	0.9775	0.9778	**0.9550**	**0.9543**	0.9700	0.9696
1000	0.9867	0.9868	0.9500	0.9488	0.9700	0.9692
10000	0.9975	0.9975	0.9400	0.9389	0.9700	0.9697

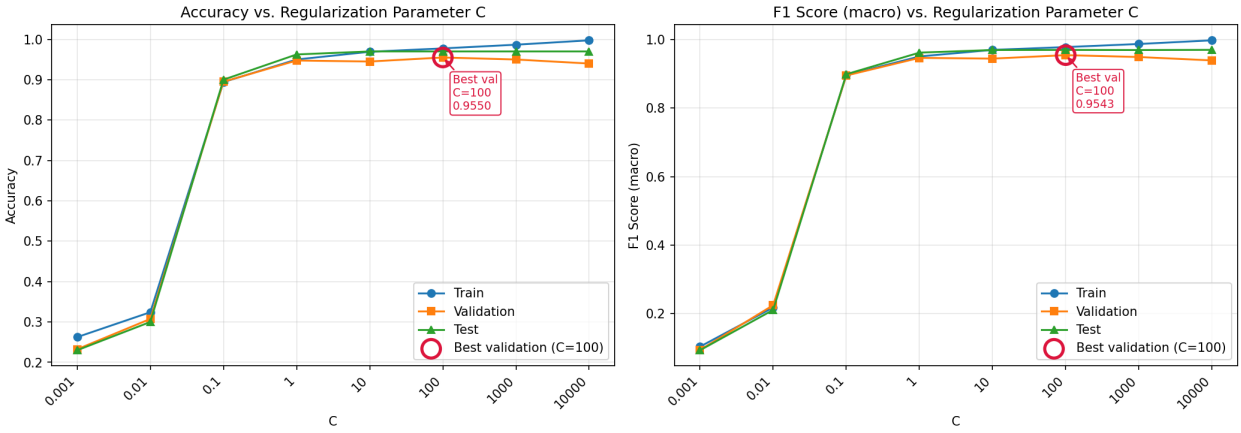


Figure 2: Accuracy and F1 score across log-spaced C values for train, validation, and test sets.

2.3 2(c) Best Generalization Performance

Best C = 100 (selected by highest validation accuracy = 95.5%).

The best parameter is selected using the validation set rather than the test set. For each candidate C, we train the SVM on the training set and compare its validation accuracy and macro-F1 score. The validation curve in Figure 2 reaches its maximum at $C = 100$, with validation accuracy 95.5% and validation macro-F1 95.4%. After $C = 100$, training accuracy continues to increase from 97.75% ($C = 100$) to 98.67% ($C = 1000$) and 99.75% ($C = 10000$), but validation accuracy drops from 95.5% to 95.0% and then 94.0%. This divergence indicates that larger C values fit the training data more tightly without improving generalization. Therefore, $C = 100$ is chosen as the best parameter before checking the final test-set result.

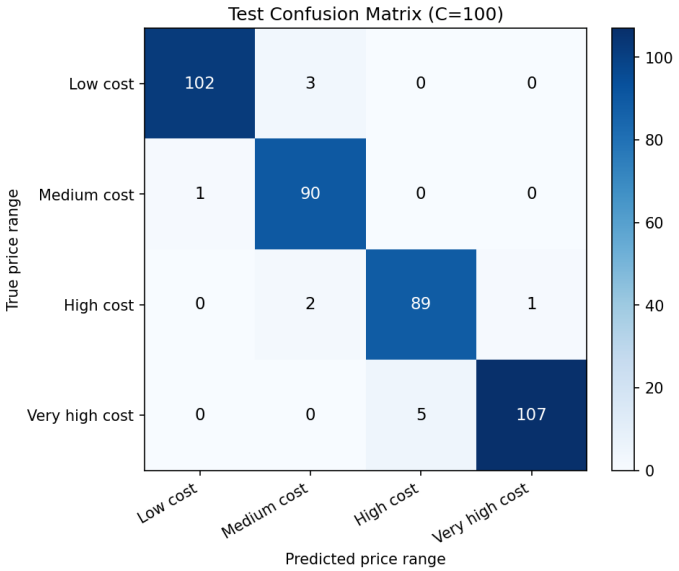


Figure 3: Test confusion matrix for the validation-selected SVM ($C = 100$), using price-range class names from low cost to very high cost. Most errors occur between adjacent price ranges, with no extreme confusion between the lowest and highest classes.

- **Underfitting at small C:** With $C \leq 0.01$, the SVM's decision boundary is overly constrained, yielding accuracy near random chance (25% for 4 classes). The margin is too wide and misclassifies most samples.
- **Training improves after C=100, but validation declines:** Increasing C beyond 100 weakens regularization, so the classifier fits the training data more aggressively. This raises training accuracy, but validation accuracy decreases, meaning the extra flexibility does not transfer to unseen validation samples. This widening train–validation gap is a classic overfitting signal.
- **Errors are mostly ordinal-neighbor mistakes:** The confusion matrix shows only 12 test errors out of 400 samples. They occur between neighboring classes (0/1, 1/2, 2/3), which is expected because `price_range` is an ordered label.

3 Q3: Association Rule Mining

We filter samples with `price_range == 1` (500 samples) and select 4 features: `ram`, `int_memory`, `px_width`, `battery_power`. Each feature is discretized into low/medium/high using a 3:4:3 **value**

range split:

$$\text{low} \leq \min + 0.3 \times \text{range}, \quad \text{medium} \leq \min + 0.7 \times \text{range}, \quad \text{high} > \min + 0.7 \times \text{range}$$

The discretized features are converted to a boolean one-hot transaction matrix (500×12).

3.1 3(a) Frequent Patterns (support ≥ 0.3)

FP-Growth finds 8 frequent patterns:

Table 5: Frequent itemsets with support ≥ 0.3

Itemset	Support
{ram_medium}	0.682
{px_width_medium}	0.416
{battery_power_medium}	0.414
{int_memory_medium}	0.412
{battery_power_medium, ram_medium}	0.318
{int_memory_low}	0.316
{battery_power_low}	0.308
{ram_medium, px_width_medium}	0.306

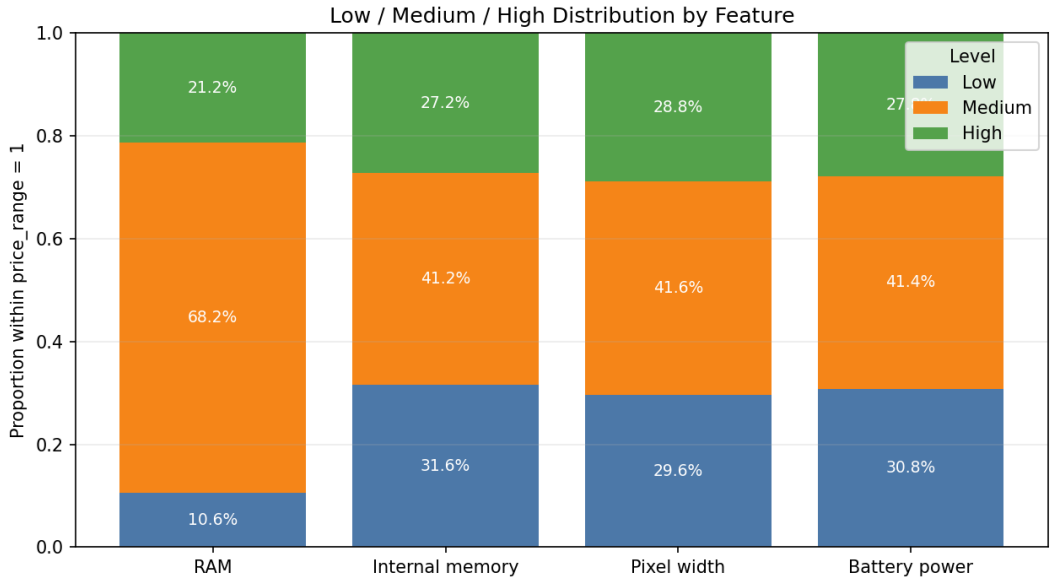


Figure 4: Low, medium, and high distribution by feature within `price_range = 1`. Each stacked bar sums to 1.0. The medium bin is the largest segment for all four selected features, supporting the observation that this price tier is dominated by medium-level specifications.

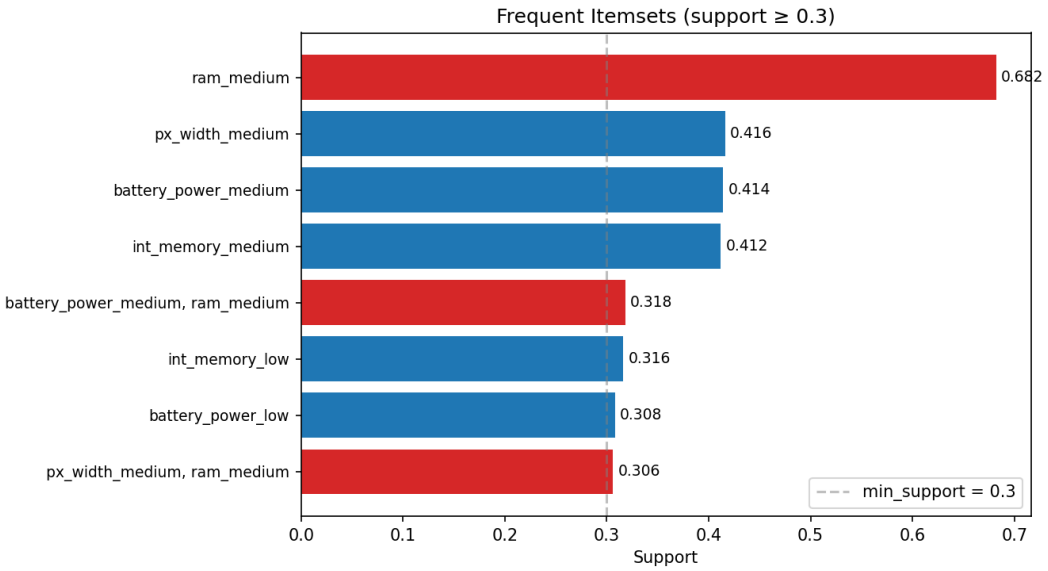


Figure 5: Support of frequent itemsets. Items involving `ram_medium` (red) dominate.

3.2 3(b) Association Rules (support $\geq$ 0.3, confidence $\geq$ 0.4, lift $\geq$ 0.8)

Table 6: Association rules meeting all criteria

Antecedent	Consequent	Support	Confidence	Lift
ram_medium	battery_power_medium	0.318	0.466	1.126
battery_power_medium	ram_medium	0.318	0.768	1.126
ram_medium	px_width_medium	0.306	0.449	1.079
px_width_medium	ram_medium	0.306	0.736	1.079

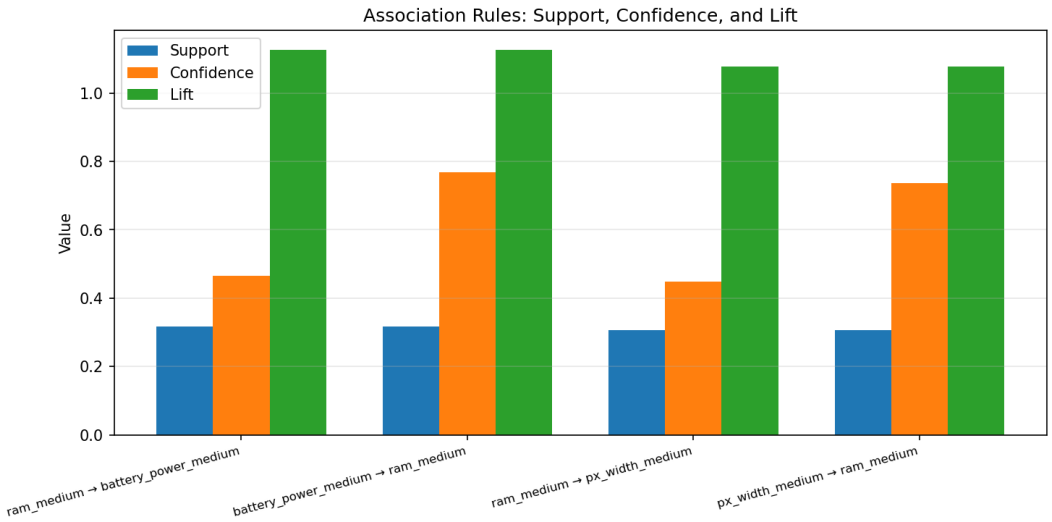


Figure 6: Support, confidence, and lift for each association rule. The asymmetric confidence between forward and reverse rules is clearly visible.

3.3 3(c) Observations

- **Medium specs dominate:** Figure 4 shows that the medium bin is the largest segment for all four selected features within price_range=1. In particular, 68.2% of price_range=1 phones have mid-range RAM, making `ram_medium` the most frequent single item and the anchor of both 2-item frequent patterns.
- **Asymmetric confidence:** `battery_power_medium` $\rightarrow$ `ram_medium` has 76.8% confidence, while the reverse has only 46.6%. This asymmetry arises because ram_medium has much higher base support (68.2%), so knowing a phone has medium battery power strongly predicts medium RAM, but not vice versa.
- **Mild positive associations:** All lift values are close to 1.0 (1.08–1.13), indicating that while medium-range features tend to co-occur slightly more than expected by chance, the associations are not dramatically strong. Within this price tier, the four features are somewhat independent.
- Under the assignment thresholds (support ≥ 0.3, confidence ≥ 0.4, lift ≥ 0.8), only 2 two-item patterns and 4 rules are retained. This is consistent with the relatively concentrated mid-range specifications in this price tier, though the high support threshold also limits the number of discoverable patterns.

4 Q4: PCA and K-Means Clustering

All 20 features from `mobile_price.csv` are standardized using z-score (`StandardScaler`). The target variable `price_range` (4 classes) is used only for evaluation, not for clustering.

4.1 4(a) Standardization

After `StandardScaler`, all features have mean ≈ 0 and std $= 1$. To visualize this transformation, Figure 7 compares three representative continuous features: `battery_power`, `px_width`, and `ram`.

Before standardization, the features are measured on different raw scales; after standardization, they are centered around 0 with comparable spread, so Euclidean-distance methods such as PCA and K-Means are not dominated by feature units.

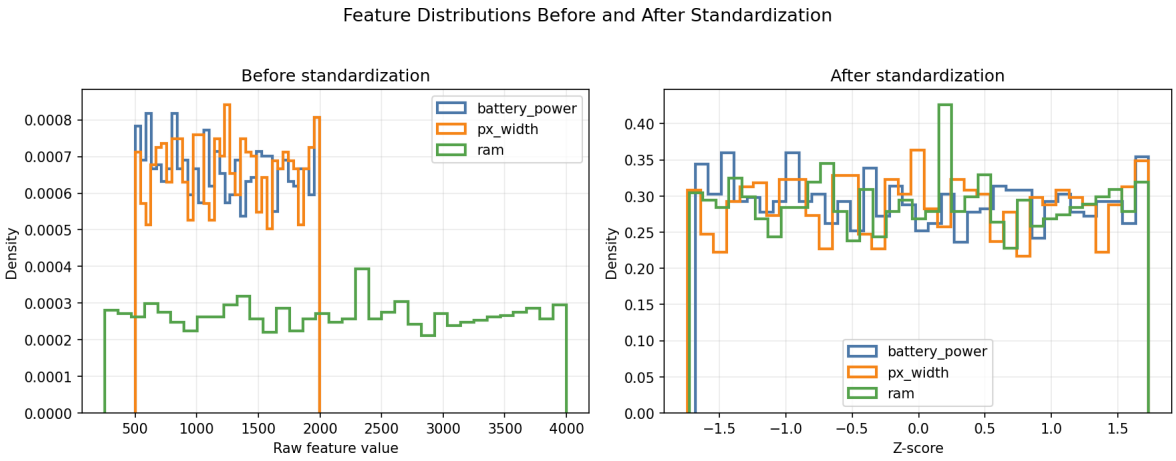
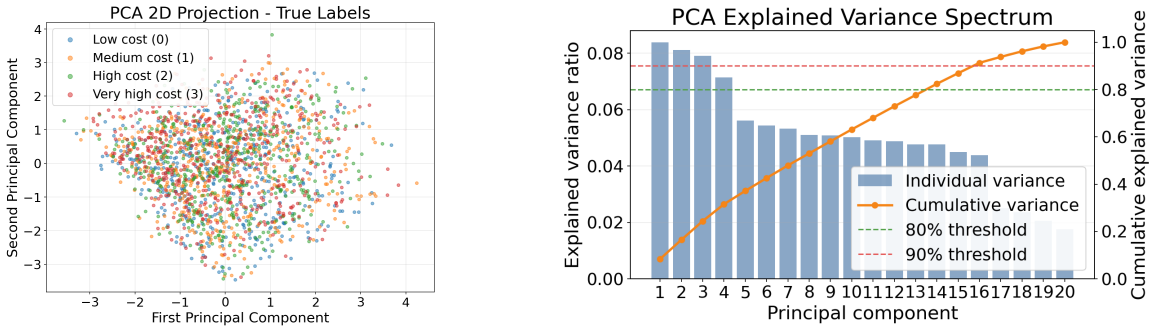


Figure 7: Distribution of three representative features before and after z-score standardization. The left panel shows raw feature values, and the right panel shows standardized z-scores.

4.2 4(b) PCA 2D Projection with True Labels



(a) PCA 2D projection with true labels.

(b) Explained-variance spectrum.

Figure 8: PCA diagnostics. The left panel shows that the first two PCs explain only 16.5% of total variance and the four price-range classes overlap heavily. The right panel shows that variance is broadly distributed across components: the first 5 PCs explain 37.2%, the first 10 explain 63.1%, 14 PCs are needed to reach 80%, and 16 PCs are needed to reach 90%.

4.3 4(c) K-Means on All Standardized Features

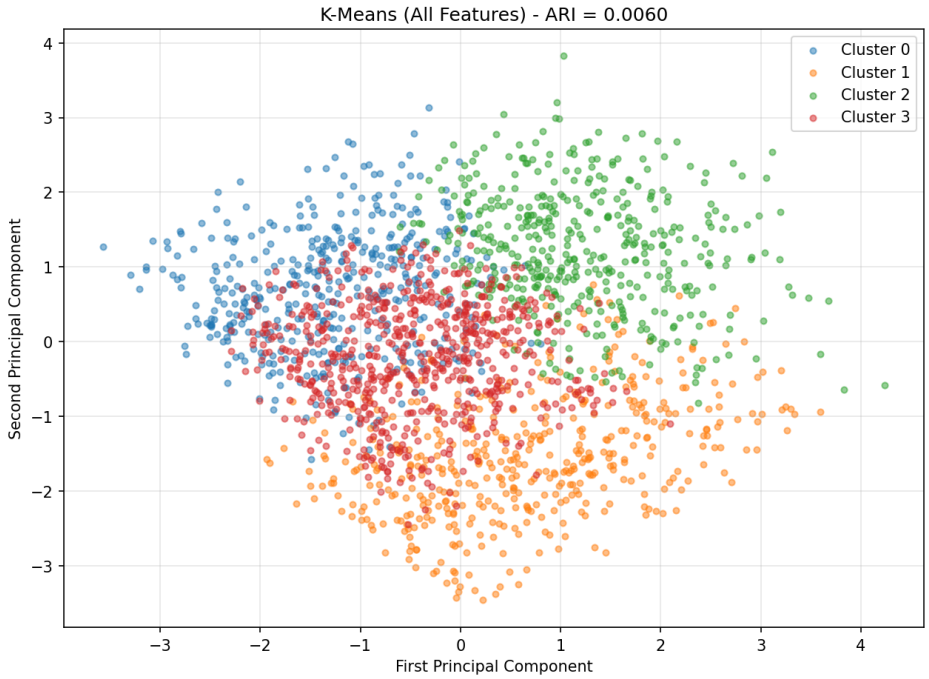


Figure 9: K-Means (all 20 features) visualized on PCA 2D plane. ARI = 0.0060.

4.4 4(d) K-Means on PCA 2D Features

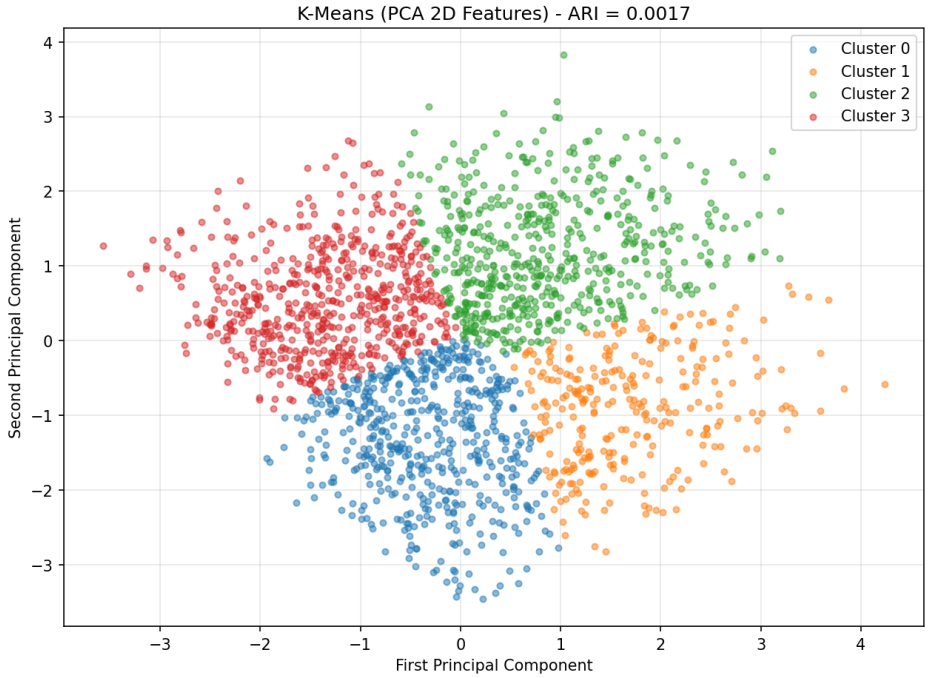


Figure 10: K-Means on PCA 2D features. ARI = 0.0017.

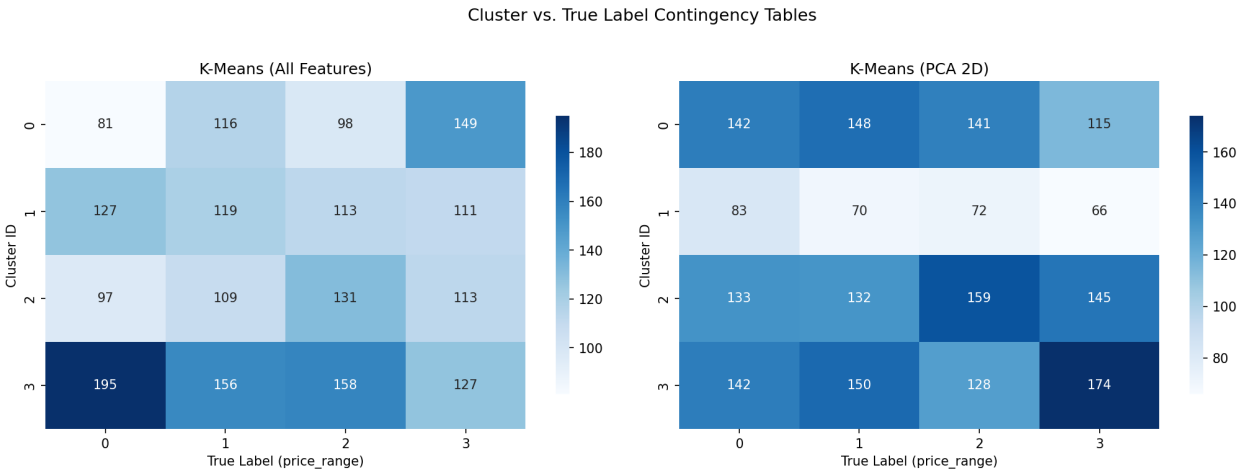


Figure 11: Contingency tables: cluster assignment vs. true label. Both heatmaps show diffuse distributions across all classes, confirming that K-Means clusters do not align with price-range labels.

4.5 4(e) Observations

- **PCA captures very little variance in two dimensions:** The first two principal components explain only 16.5% of total variance. The full spectrum in Figure 8 confirms that the information is broadly distributed: even 10 PCs retain only 63.1% variance, and 14 PCs are needed to reach 80%. Therefore the 2D projection is useful for visualization but too compressed to preserve the full structure.
- **Both K-Means approaches fail:** ARI values of 0.0060 (all features) and 0.0017 (PCA 2D) are near zero, meaning the clustering is essentially random with respect to the true labels.
- **Dimensionality reduction hurts slightly:** K-Means on all 20 features (ARI=0.0060) marginally outperforms K-Means on 2D PCA (ARI=0.0017), likely consistent with the information loss from reducing to 2D (which retains only 16.5% of total variance), though PCA variance retention and class separability are not the same thing.
- **K-Means fails to recover label-consistent partitions:** K-Means assumes spherical, equally-sized clusters in Euclidean space, which does not match the structure of this dataset. While supervised SVM (Q2) achieves >95% accuracy by exploiting label information to learn non-linear decision boundaries, K-Means — as an unsupervised method — cannot leverage labels and relies solely on Euclidean distance, which is insufficient to separate these price-range classes. This is not an apples-to-apples comparison, but it highlights that label information is critical for this classification task.

5 Q5: Enhancing K-Means with Lift-Sum Feature Weighting

5.1 Motivation

The Q4 result shows the core problem: standard K-Means gives every standardized feature equal weight in Euclidean distance, yet its clusters are almost unrelated to the true `price_range` labels (ARI = 0.0058 on average). This suggests that equal weighting lets weak or irrelevant features dilute the few features that actually carry price-range signal.

The Q2 RBF-SVM provides a supervised reference for this feature-importance imbalance. Because the validation-selected RBF-SVM ($C = 100$) has no direct linear coefficient vector, we use permutation importance on the Q2 test set: each feature is shuffled repeatedly, and the resulting drop in test accuracy is measured. Figure 12 shows that `ram` dominates the SVM's prediction behavior, followed by `battery_power`, `px_width`, and `px_height`. This motivates replacing equal K-Means distances with a feature-weighted distance.

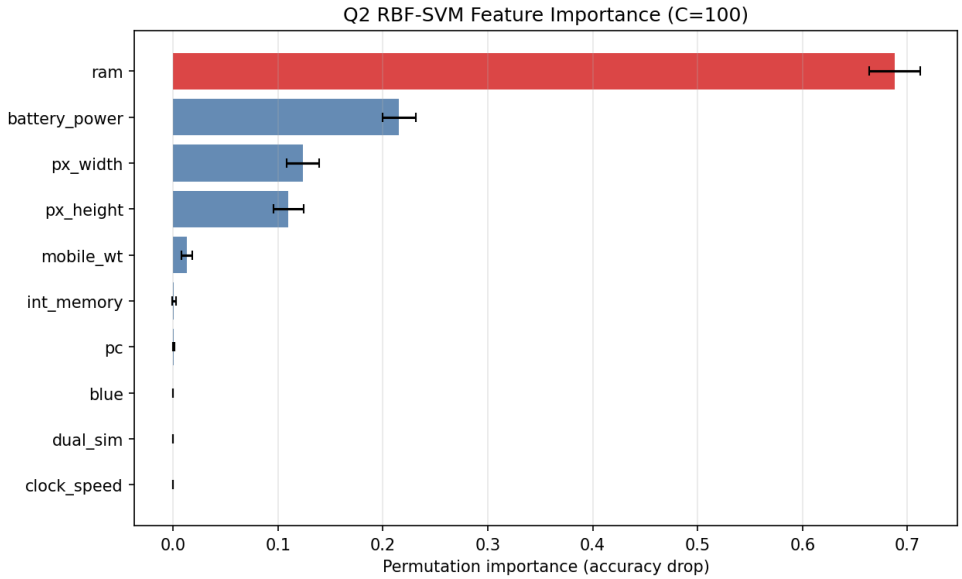


Figure 12: Permutation importance of the Q2 RBF-SVM ($C = 100$). The largest accuracy drop occurs when `ram` is permuted, showing that the supervised model relies on highly uneven feature importance rather than equal feature contribution.

5.2 Lift-Sum Feature Importance

The rewritten Q5 uses association-rule lift as a second feature-importance signal. The method is **semi-supervised**: `price_range` labels are used during class-stratified rule mining, while the final K-Means clustering step itself remains unsupervised.

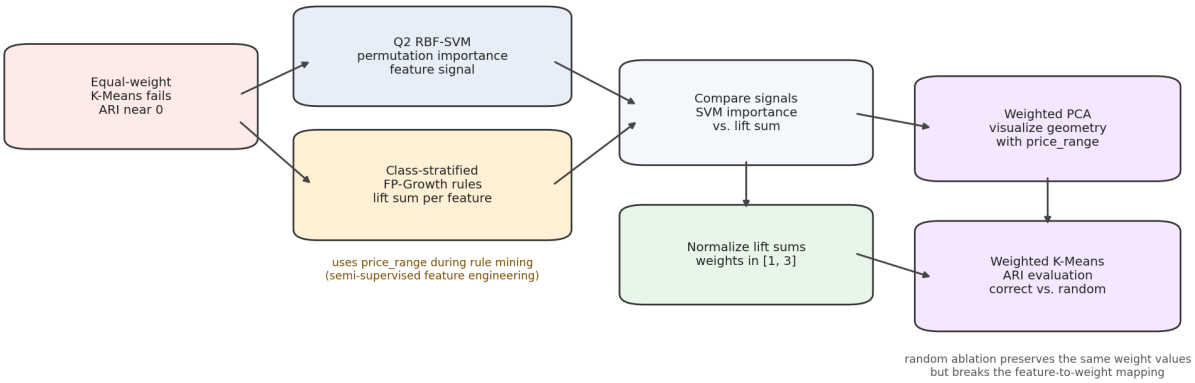


Figure 13: Rewritten Q5 method design. Equal-weight K-Means motivates feature weighting; Q2 SVM permutation importance provides a supervised reference; class-stratified FP-Growth produces lift sums; normalized lift sums are used as K-Means feature weights; random reassignment tests whether the feature-to-weight mapping matters.

Each original feature is discretized with the same 3:4:3 range split used earlier:

$$\begin{aligned} \text{low} &: [\min, \min + 0.3\,\text{range}], \\ \text{medium} &: (\min + 0.3\,\text{range}, \min + 0.7\,\text{range}], \\ \text{high} &: (\min + 0.7\,\text{range}, \max]. \end{aligned}$$

Then FP-Growth is run separately inside each `price_range` class using `min_support=0.3`. Association rules are retained when confidence ≥ 0.5 and lift ≥ 1.0. For each original feature f, we sum the lift values of all retained rules that contain any discretized item derived from f:

$$\text{LiftSum}(f) = \sum_{r:\, f \in r} \text{lift}(r).$$

The lift sums are converted to feature weights in the range $[1, 3]$:

$$w_f = 1 + 2 \cdot \frac{\text{LiftSum}(f)}{\max_g \text{LiftSum}(g)}.$$

The highest lift-sum features are `ram` (295.34, weight 3.00), `three_g` (259.75, weight 2.76), `fc` (193.09, weight 2.31), `four_g` (118.57, weight 1.80), and `px_height` (54.23, weight 1.37).

5.3 SVM vs. Lift-Sum Comparison

Figure 14 compares normalized SVM permutation importance and normalized lift sum. The agreement is meaningful but not perfect. Across all 20 features, Pearson correlation is moderate ($r = 0.535$, $p = 0.015$), while rank correlation is weak (Spearman $\rho = 0.156$, $p = 0.510$). When binary features are excluded, Pearson correlation rises to $r = 0.744$ ($p = 0.002$), indicating that lift sums are more aligned with the SVM signal among the continuous and non-binary features.

The strongest agreement is the most important one: both methods identify `ram` as the dominant feature.

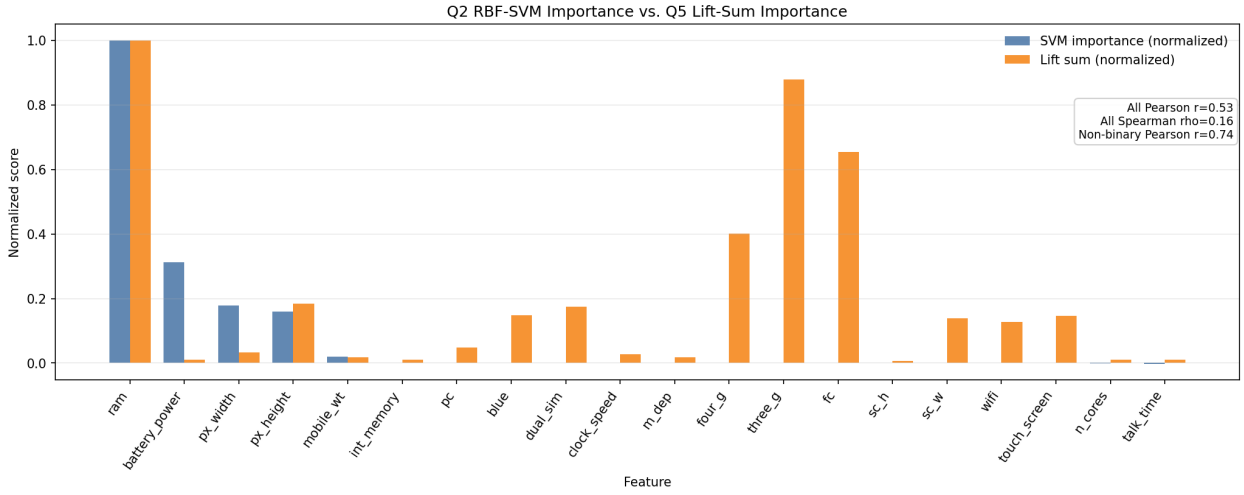


Figure 14: Bar-plot comparison between Q2 RBF-SVM permutation importance and Q5 class-stratified lift sums. The x-axis lists the original features, and the two bars for each feature show normalized SVM importance and normalized lift sum. The relationship is evidence of partial alignment, especially for major continuous signals, but not perfect rank agreement across all features.

5.4 Weighted PCA Analysis

To visualize the effect of lift-derived weights, we multiply the standardized feature matrix by the lift weights and then run PCA:

$$X_{\text{weighted}}[:, f] = X_{\text{scaled}}[:, f] \cdot w_f.$$

Figure 15 shows the PCA projection after weighting. The first two weighted PCs explain 39.2% of the variance, compared with only 16.5% for the original 2D PCA in Q4. The price-range ordering becomes more visible, especially along the weighted PCA bands, although the classes still overlap. This supports the claim that lift-based weighting improves the geometry, but does not make the problem linearly separable.

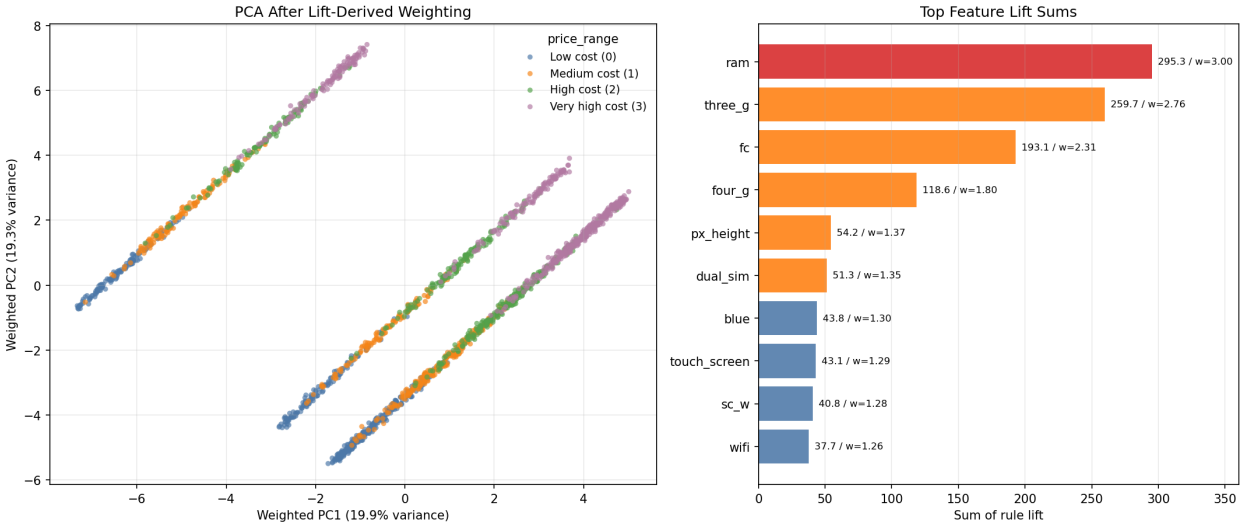


Figure 15: PCA 2D projection after applying class-stratified lift-derived feature weights, with the top feature lift sums shown beside it. PCA is fitted on weighted standardized features; true `price_range` labels are shown only for interpretation.

5.5 Randomization Ablation

The ablation study tests whether improvement comes from the correct feature-to-weight mapping or merely from having an uneven set of weights. The correct method assigns each lift-derived weight to its original feature. The random baseline preserves exactly the same weight values, but randomly permutes them across features before running K-Means. We use 20 random permutations and 5 K-Means seeds {0, 10, 42, 100, 999}, yielding 100 random evaluations.

Table 7: Randomization ablation for lift-derived feature weighting

Method	Evaluations	ARI mean ± std	Mean accuracy
Baseline equal-weight K-Means	5	0.0058 ± 0.0006	0.2962
Correct lift-weighted K-Means	5	**0.1769 ± 0.0009**	**0.4421**
Randomly reassigned lift weights	100	0.0497 ± 0.0965	0.3168

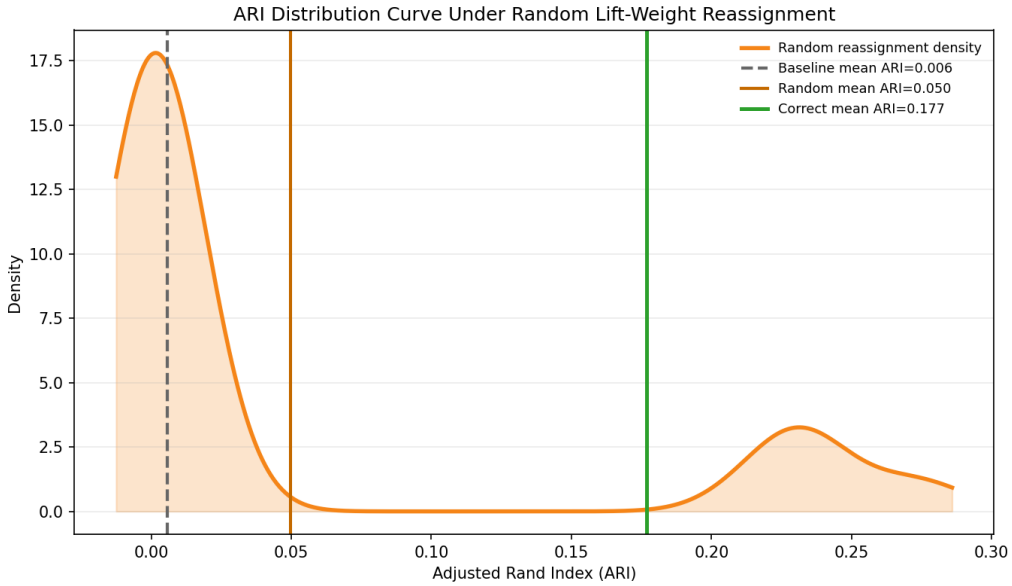


Figure 16: ARI distribution curve for the 100 random lift-weight reassignments. The vertical reference lines show the mean ARI of equal-weight K-Means, the mean ARI of the random reassignment distribution, and the mean ARI of the correct lift-weighted mapping.

The correct lift-weighted method reaches ARI = 0.1769, far above the equal-weight baseline (0.0058). Random reassignment averages ARI = 0.0497, which is better than baseline in some random trials because the weight distribution is still uneven, but it remains far below the correct assignment. This supports the main claim: the improvement is not only caused by using larger and smaller weights, but by assigning the high lift-derived weights to the features where they belong.

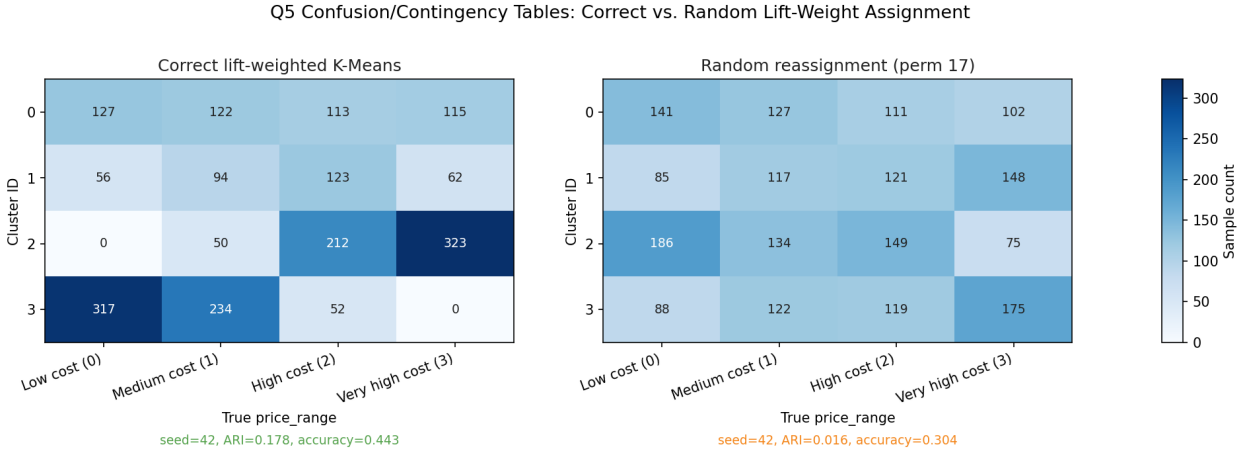


Figure 17: Confusion/contingency tables for the correct lift-weighted K-Means and a representative random reassignment. The representative random permutation is chosen as the permutation whose mean ARI is closest to the mean random-permutation ARI. Both panels use `random_state=42`. The correct mapping produces more label-concentrated clusters, while random reassignment remains diffuse.

5.6 Conclusion

Q5 therefore changes from a broad feature-engineering experiment into a focused feature-weighting argument. Equal-weight K-Means fails because it treats all standardized features as equally useful. Q2 SVM permutation importance confirms that the actual prediction signal is highly uneven. Class-stratified association-rule lift sums recover part of this feature-importance structure, most clearly for `ram` and several non-binary features. Applying these lift-derived weights improves both the PCA geometry and K-Means ARI, and the randomization ablation shows that the correct feature-to-weight assignment is essential.